Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Condition and Loop

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

CONTENTS

01 Conditional Statements

02 Loop Statements

03 Practice

아주대학교
AJOU UNIVERSITY

01

Conditional Statements

Control Flow

- **Sequential:** Statements execute from top to bottom one by one.
- **Conditional or selection:** Out of two instructions, only one will be executed successfully based on the specified condition. This is because the condition generates the result as either true or false.
- **Repetition or loop:** Group of statements repeat whenever the specified condition is true.

Control Flow

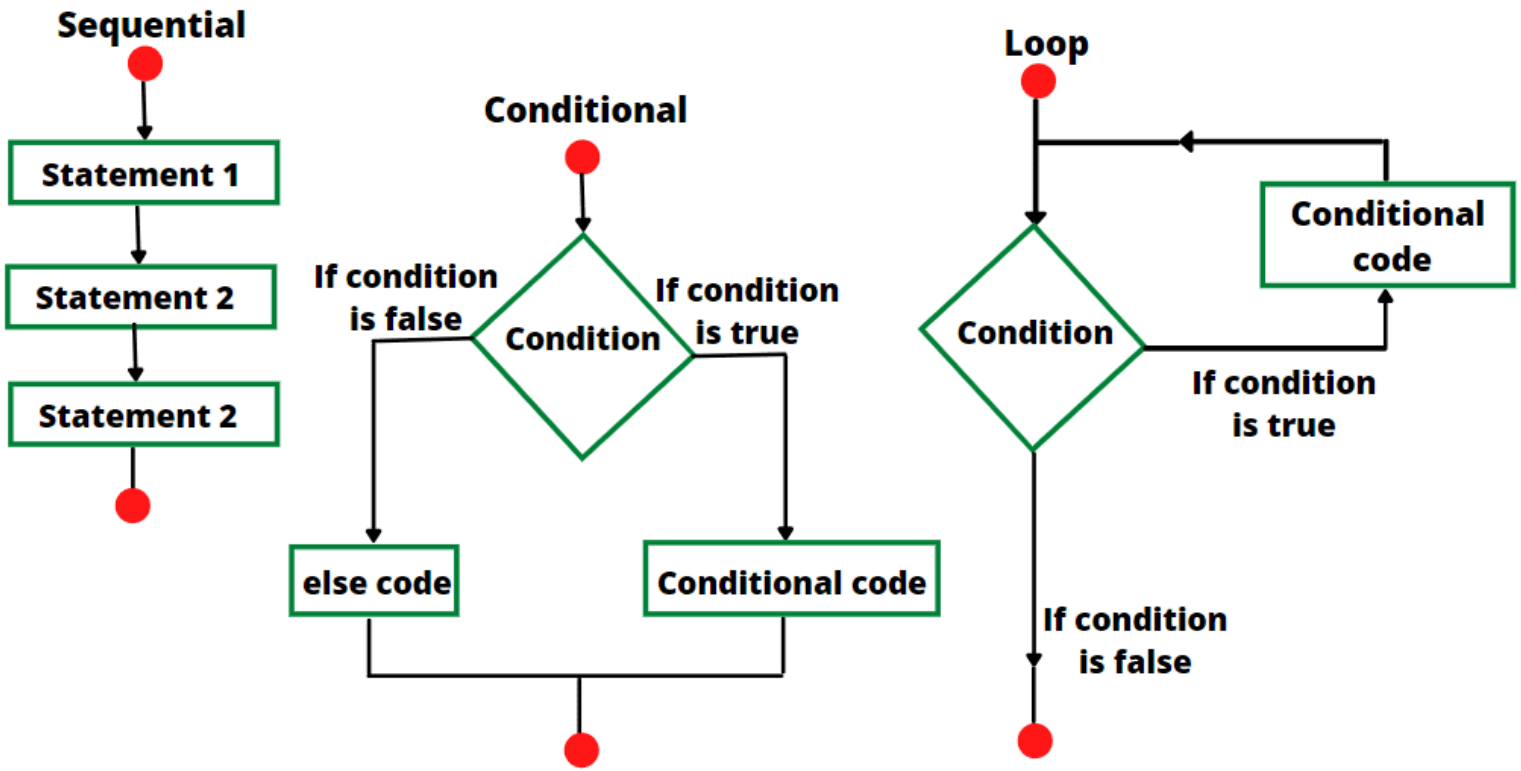


Fig: Three main types of control flow in computer program

Control Flow

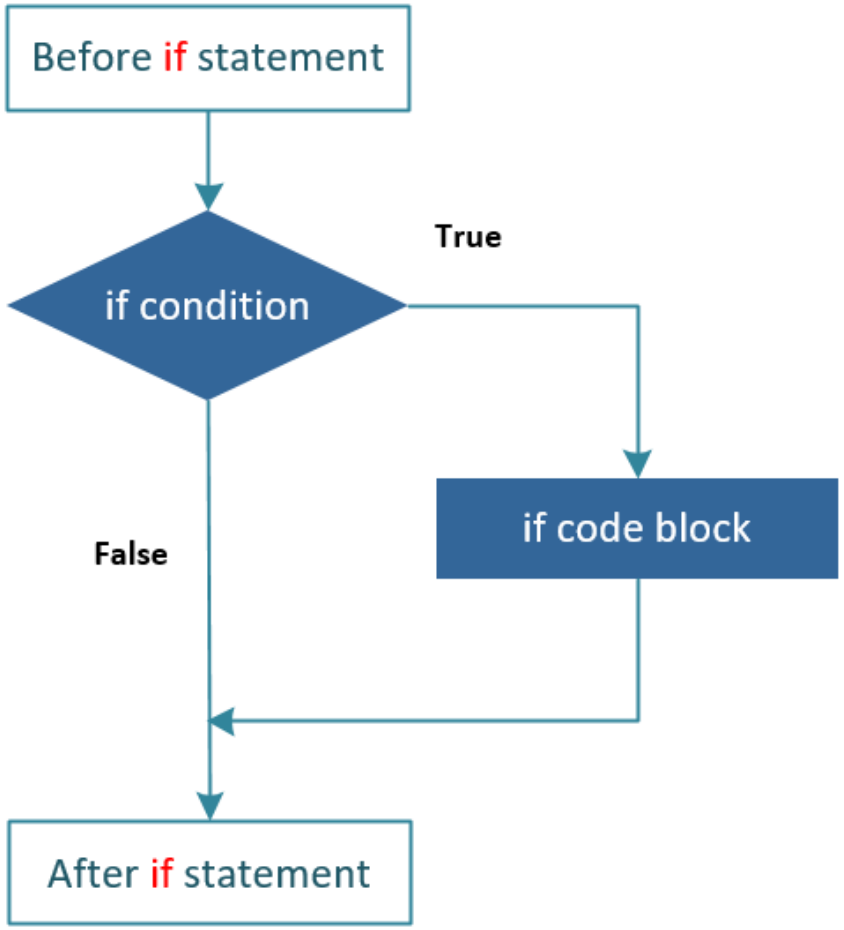
- Conditional statements
  - Used to execute commands only when a specific condition is met.
  - The most common conditional statement is the “**if**” statement.
  - **“if”** statement format:
    - If(condition) { code block }
    - If(condition) { code block } else { code block }
    - If(condition) { code block } else if(condition) {code block} else { code block }

Conditional statements

- An **if else** in Java is a two-way conditional statement that decides the execution path based on whether **the condition is true or false**.
- In other words, **if-else statement is used to perform a specific action (task) depending on whether a specified condition is true or false**. Here, an action signifies a statement or a group of statements.
- A **one-way if statement** executes a statement if and only if the specified condition is true. **If the condition is false, nothing will be done**. But, suppose we want to take alternative actions when the specified condition is false. In this case, we will use a **two-way if-else statement**.

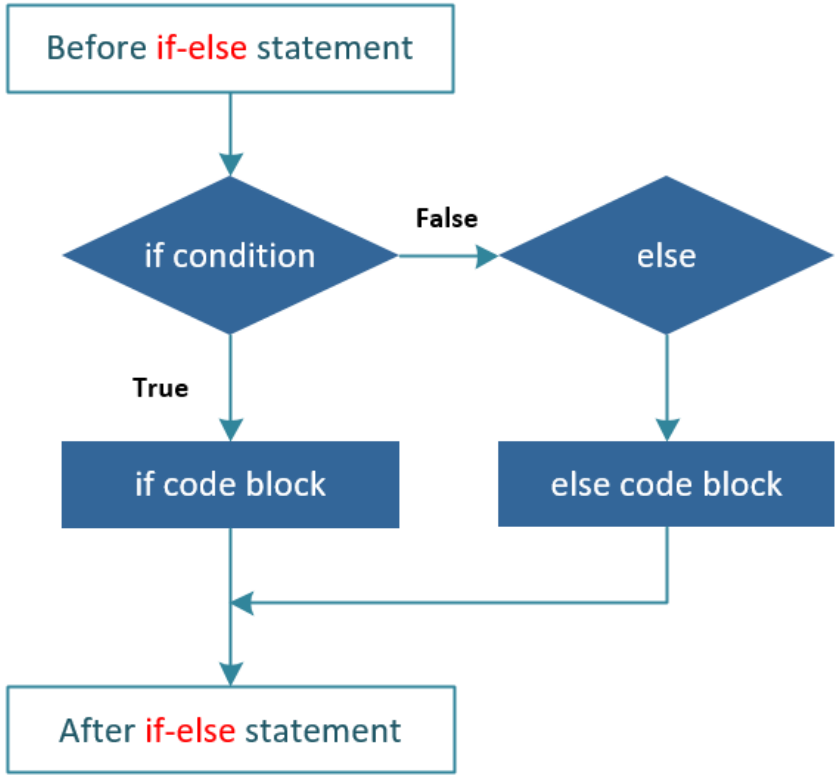
Conditional statements

- **“if”** statements



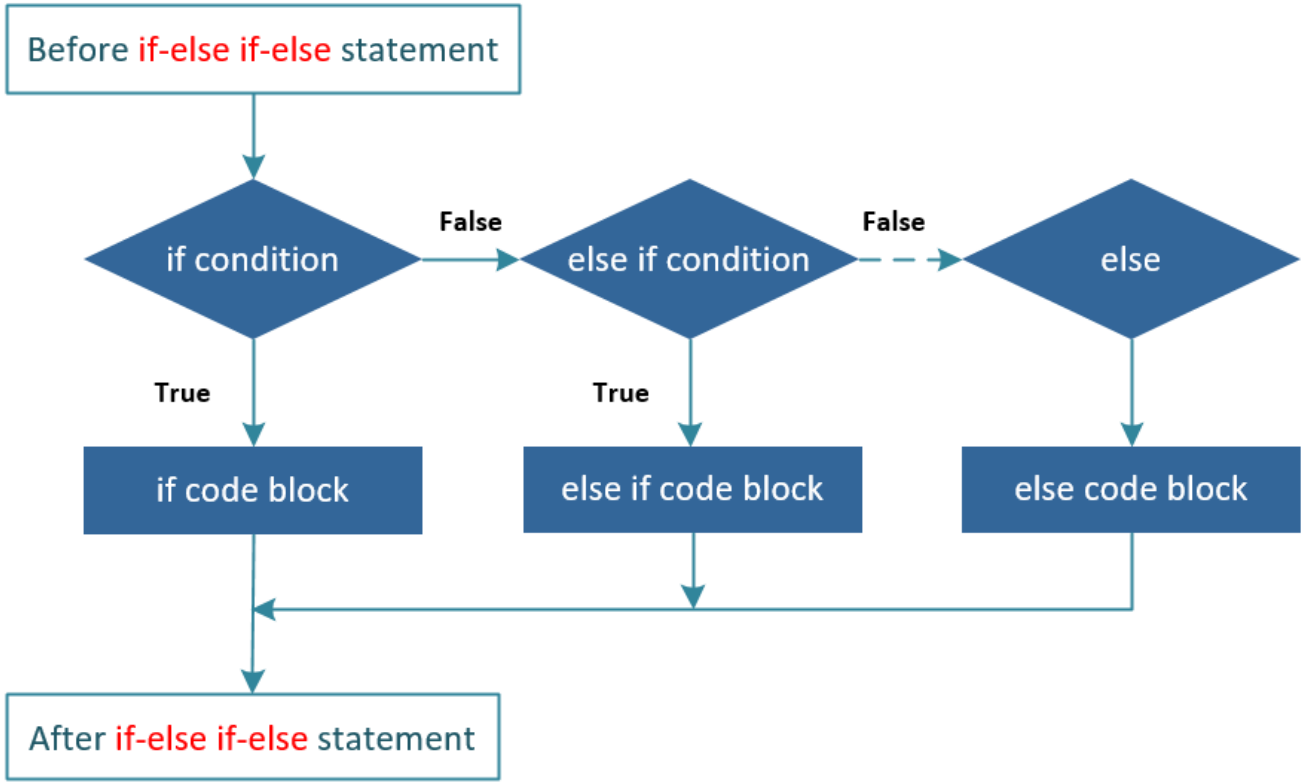
Conditional statements

- **“if…else…”** statements



Conditional statements

- **“if…else if…else”** statements



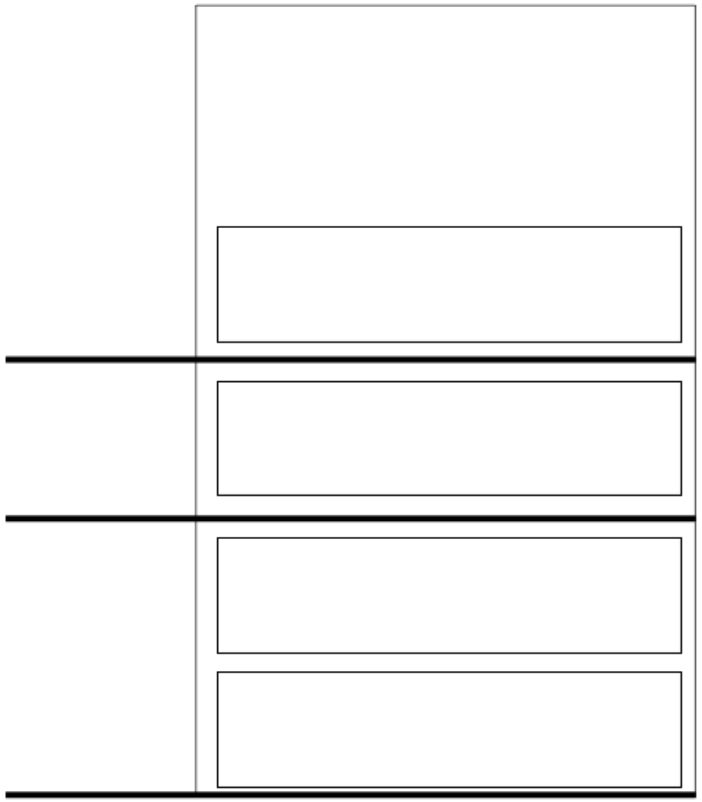
Scope

- _ifAndScope

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner((System.in));
        System.out.println("please input age: ");
        int age = s.nextInt();

        if(age>20){
            System.out.printf("Age before treatment is %d years old\n", age);
            age = 20;
        }

        System.out.println(("Age after treatment is " + age + "years old"));

    }
}
```

Scope

- _ifAndScope



```
{
    int a;
    int b;
    {
        int c;
        {
            int d;
        }
    }
}
```

Conditional statements

- _maxiumVal

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        int input = s.nextInt();
        int max = input;

        input = s.nextInt();
        if(input > max) max = input;

        input = s.nextInt();
        if(input > max) {
            max = input;
        }

        System.out.println("MAX: " + max);
    }
}
```

Conditional statements

- _busFare

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        int age = s.nextInt();
        int fee = 750;

        if(age >= 20) {
            age = 20;
            fee = 1000;
        }

        System.out.println("Age:" + age + ", Fare: " + fee);
    }
}
```

Conditional statements

- _ifElseSample

```
import java.util.Scanner;

public class Main {
	public static void main(String[] args) {
		Scanner s = new Scanner(System.in); int input = s.nextInt();

		//10_condiSample
		//int result = input <= 10 ? 10 : 20; int result = 0;
		if(input <= 10)
			result = 10;
		else
			result = 20; System.out.println("result: " + result);
	}
}
```

Conditional statements

- _classify01

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Score: ");
        int score = s.nextInt();
        char ch = 'A';

        if(score >= 90)
            ch = 'A';
        else if (score >= 80)
            ch = 'B';
        else if (score >= 70)
            ch = 'C';
        else if (score >= 60)
            ch = 'D';
        else
            ch = 'F';

        System.out.println("Score: " + score + " Grade: " + ch);
    }
}
```

Conditional statements

- _classify02

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("점수: ");
        int score = s.nextInt();
        char ch = 'A';

        if(score >= 80) {
            if(score >= 90)
                ch = 'A';
            else
                ch = 'B';
        }
        else {
            if (score >= 70)
                ch = 'C';
            else {
                ch = 'D';
                if (score < 60)
                    ch = 'F';
            }
        }

        System.out.println("점수: " + score + " 학점: " + ch);
    }
}
```

switch statements

- A **switch statement** in Java is a conditional control statement (or multi-way decision statement) that executes one statement from multiple conditions. It uses the result of an expression to evaluate which statements to execute. It is an alternative to if-else-if ladder statement.
- In other words, a switch statement executes statements based on the value of a variable or an expression, against a list of case values. If a match is found, a block of statements corresponding to that case is executed.

switch statements

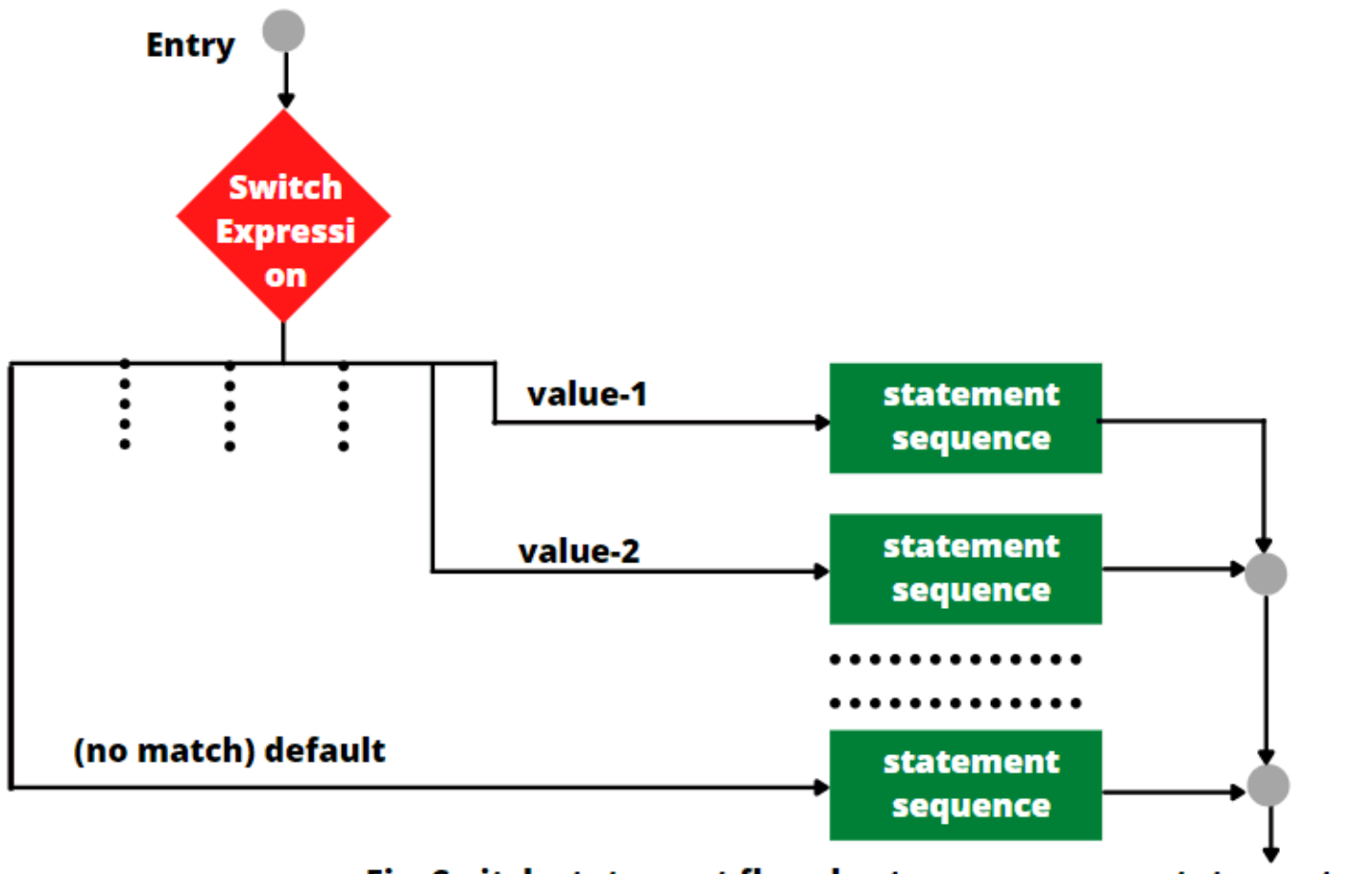


Fig: Switch statement flowchart

switch statements

- _switchCase01

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        int left = s.nextInt();
        char operator = s.next().charAt(0);
        int right = s.nextInt();
        int result = 0;

        switch (operator)
        {
            case '+':
                result = left + right;
                break;

            case '-':
                result = left - right;
                break;

            case '*':
                result = left * right;
                break;

            case '/':
                result = left / right;
                break;
        }

        System.out.println("result: " + result);
    }
}
```

switch statements

- _switchCase02

```
Scanner s = new Scanner(System.in); int score = s.nextInt();
char ch = 'A';

switch (score / 10) { case 10:
    case 9:
        ch = 'A';
        break;

    case 8:
        ch = 'B';
        break;
    … default:
        ch = 'F';
}

System.out.println("result: " + ch);
```

Quiz

- 1. What is the main purpose of control statements that change the program's execution flow based on specific conditions?
  A. To manage variable names.
  B. To improve code readability.
  C. To execute different code blocks depending on conditions.
  D. To efficiently process comments.

Quiz

- 2. In Java, what happens to a local variable when its scope ends?
  A. It becomes accessible from anywhere in the program.
  B. It is automatically removed from memory.
  C. It can be accessed again outside of its scope.
  D. It is automatically renamed.

Quiz

- 3. What is the main execution flow difference between using multiple `if` statements sequentially and `if-else if` chains?
  A. The scope of variable declarations changes.
  B. The code's indentation rules become stricter.
  C. If-else if does not check the remaining conditions when a condition is met.
  D. The available comparison operators change.

Quiz

- 4. What happens when a "position breakpoint" is set in debugging mode?
  A. It skips the specified line of code and moves to the next line.
  B. It automatically changes the values of all variables.
  C. Program execution pauses at that line.
  D. It automatically corrects all syntax errors in the code.

Quiz

- 5. Which data type is least suitable for use as a condition value in a Java "switch-case" statement?
  A. int
  B. char
  C. String
  D. double

02

Loop Statements

Control Flow

- Loop statements:
  - Used to repeatedly execute statements.
  - **while** loop
  - **do while** loop
  - **for** loop
  - **foreach** loop
  - **break** and **continue** (Unconditional Statements)

while loop statements

- The **while loop** in Java is **the most fundamental loop statement** that repeats a statement or series of statements as long as the **specified conditional expression** evaluates to true.
- In other words, a while loop repeats the **body of the loop** as long as the loop condition holds. Each repetition of the body of loop is called **iteration**.
- While loop is the simplest of all the looping structures in Java. It is an entry-controlled loop statement. In the entry-controlled loop, the test condition evaluates first. If the specified condition is true, the body of loop executes.
- If the conditional expression does not satisfy, then the body of loop will not execute and the program will proceed to the next set of statements.

while loop statements

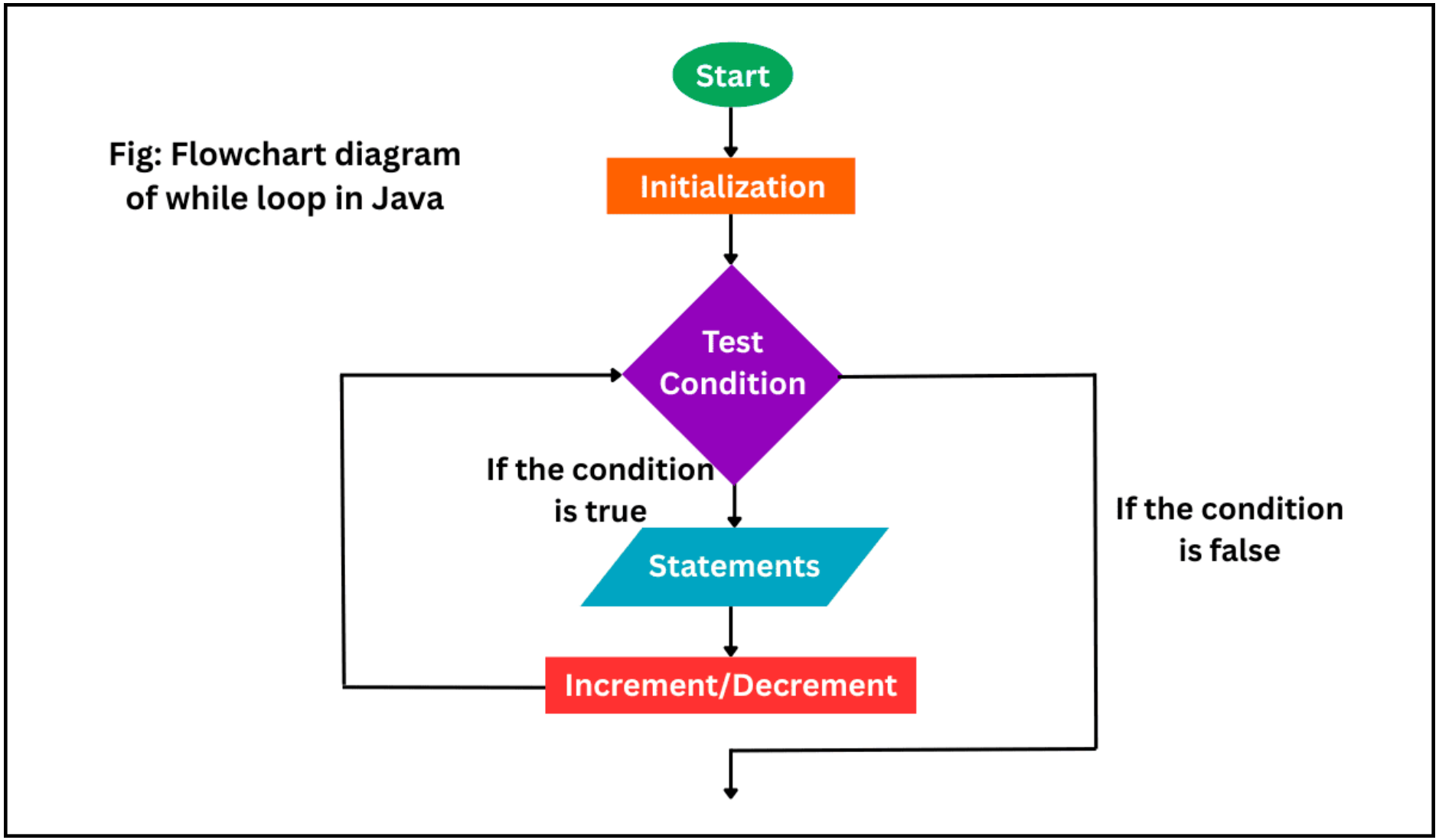


Fig: Flowchart diagram of while loop in Java

while loop statements

- A simple example program to display the numbers from 1 to 5 in Java by using while loop

```
package javaProgram;
public class WhileLoopEx {
public static void main(String[]
args)
{
  int i = 1; // Initialization.
  while(i <= 5)
  {
    System.out.println(i);
    i++; // Increment.
   }
 }
}
```

while loop statements

- _whileInput

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception {
        int input = 0;
        while((input = System.in.read()) != 'q') {
            System.out.println("input: " + input);
        }
        System.out.print("Exit");
    }
}
```

while loop statements

- _whilePrintStar

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Input number: ");
        int input = s.nextInt();

        if(input < 0)        input = 1;
        else if(input > 9)   input = 9;

        int cnt = 0; while(c
        nt < input) {
            System.out.print("*\t");
            cnt++;
        }
    }
}
```

while loop statements

- _variableInLoop

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception{
        Scanner s = new Scanner(System.in);

        int ch = 0;
        while((ch = System.in.read()) != 'q') {
            int cnt = 0;
            System.out.printf("cnd: %d, %c\n", cnt, ch);
            cnt++;
        }
    }
}
```

while loop statements

- _endlessLoop

```
public class Main {
    public static void main(String[] args) {
        int cnt = 0;
        while(cnt >= 0) {
            cnt = cnt + 1 - 1;
            //cnt++;
        }
        System.out.println("cnt: " + cnt);
    }
}
```

Because “cnt” it is an integer, it stops when the value goes out of range.

do...while loop statements

- A **do while loop** in Java is a variant form of while loop. It is the same as a while loop, except that it executes the body of the loop first and then evaluates the loop continuation condition.
- In the while loop statement, the test condition evaluates first before the loop body executes. If the condition is initially false, the body of loop will not execute at all and the control of execution will proceed to the next set of statements to execute.
- However, sometimes, it might need to execute the body of a loop at least once before the test performs. We can handle such situations with the help of do while loop in Java.
- The do while loop always executes its body at least once before the test evaluates because its test conditional expression is at the bottom of the loop. It is especially useful when we want to execute a certain block of code at least once, regardless of whether the loop condition is initially true or false.

do...while loop statements

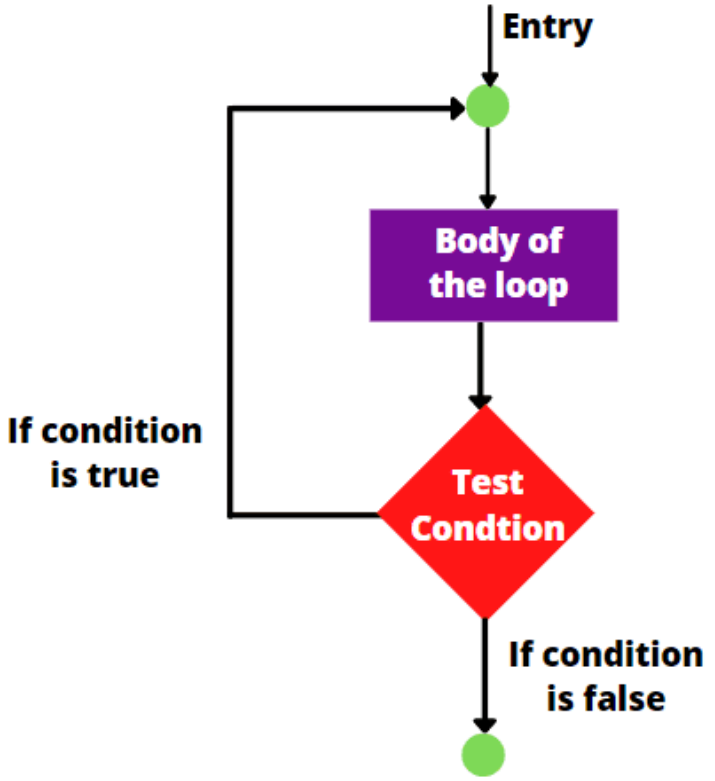


Fig: do while loop flowchart

do...while loop statements

- A simple example program where we will display numbers from 1 to 6 in Java using do while statement.

```
package javaProgram;
public class Test {
public static void main(String[] args)
{
// Initialize the counter.
   int x;
   x = 1; // starting number is 1.

// Execute the loop at least once and print the value of x.
   do {
      System.out.println(x); // print x value.
      x++; // increment x value by 1.
   } while(x <= 6); // This statement will execute as long as x <= 6.
 }
}
```

for loop statements

- The **for loop** in Java is an entry-controlled loop structure that executes a set of statements **a fixed number of times**. It is perfect for those scenarios where we know the exact number of iterations needed to accomplish a task.
- The for statement provides a more concise syntax for creating loops. It executes a block of statements as long as the condition is true.

for loop statements

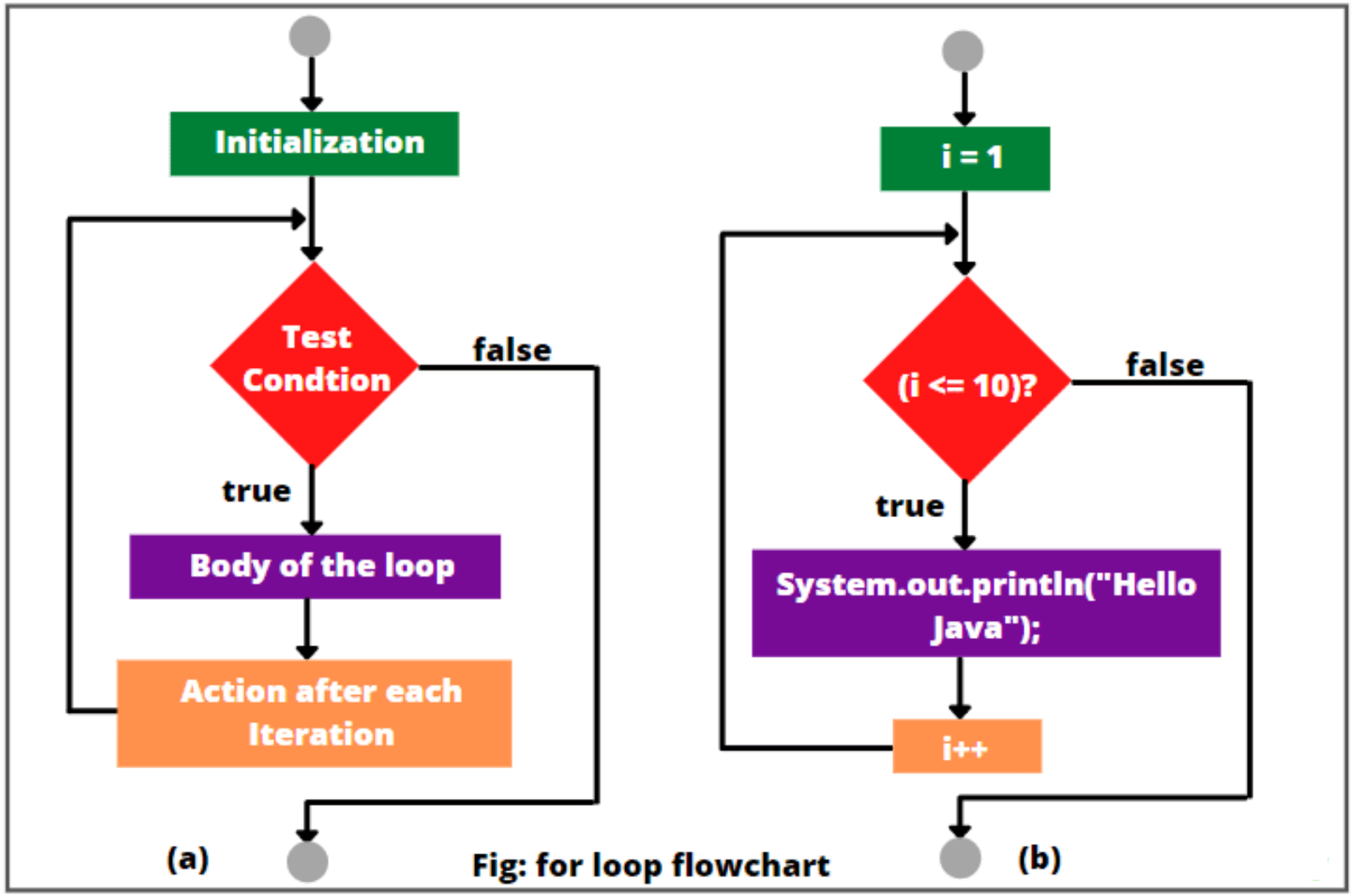


Fig: for loop flowchart

for loop statements

- A simple example program where we will display numbers from 1 to 5 and from 5 to 1 using for loop statement.

```
package javaProgram;
public class Test {
public static void main(String[] args)
{
   System.out.println("Displaying numbers from 1 to 5:");

// Applying for loop.
   for(int i = 1; i <=5; i++) { // Here, i++ is increment operator.
       System.out.println(i);
   }

   System.out.println("Displaying numbers from 5 to 1:");
   for(int j = 5; j > 0; j--) { // Here, j-- is decrement operator.
       System.out.println(j);
   }
 }
}
```

for loop statements

- _calFactorial

```
package javaProgram;
public class FactorialEx {
public static void main(String[] args)
{
  int number = 5;
  int factorial = 1;

  for (int i = 1; i <= number; i++) {
                factorial *= i;
   }
  System.out.println("Factorial of " + number + " is: " + factorial);
 }
}
```

for loop statements

- _endlessLoop

```
package javaProgram;
public class Test {
public static void main(String[] args)
{
  int i = 1;
  int sum = 0;

// Infinite for loop
   for(; ;) {
      sum = sum + i * i * i;
      i++;
      if(i >= 5) break; // If the i value exceeds 5, then come out of this loop.
   }
   System.out.println("Sum: " +sum);
 }
}
```

for loop statements

- _initTwoVar

```
package javaProgram;
public class Test {
public static void main(String[] args)
{
  int i, j;
  for(i = 1, j = 5; i <=  5; i++, j--)
  {
     System.out.println(i+ "₩t" +j);
  }
 }
}
```

for loop statements

- _compoundRelation

```
package javaProgram;
public class Test {
public static void main(String[] args)
{
    int x = 1, sum = 0;
    for(x = 1; x < 20 && sum < 20; x++) {
        sum = sum + x;
    }
   System.out.println("Sum: " +sum);
  }
}
```

Nesting of loops

- _nestedFor

```
public class Main {
    public static void main(String[] args) {
        for(int i = 0; i < 5; ++i) {
            for(int j = 0; j < 5;  ++j)
                System.out.print("*\t");
            System.out.print('\n');
        }
    }
}
```

Break Statement

- A **break statement** in Java is **used to break a loop or switch statement**. When a break statement encounters inside a loop statement, **the loop immediately ends at a specified condition**. As a result, the program control continues the execution of the next statement immediately following the loop body.
- Similarly, when a break statement executes inside the switch block, it terminates ‘case’ inside the switch block and stops the execution of more cases inside the switch.
- **When the loops are nested, the break statement breaks only the inner loop.** We can use break statement in all types of loops such as for loop, while loop, and do-while loop.

Break Statement

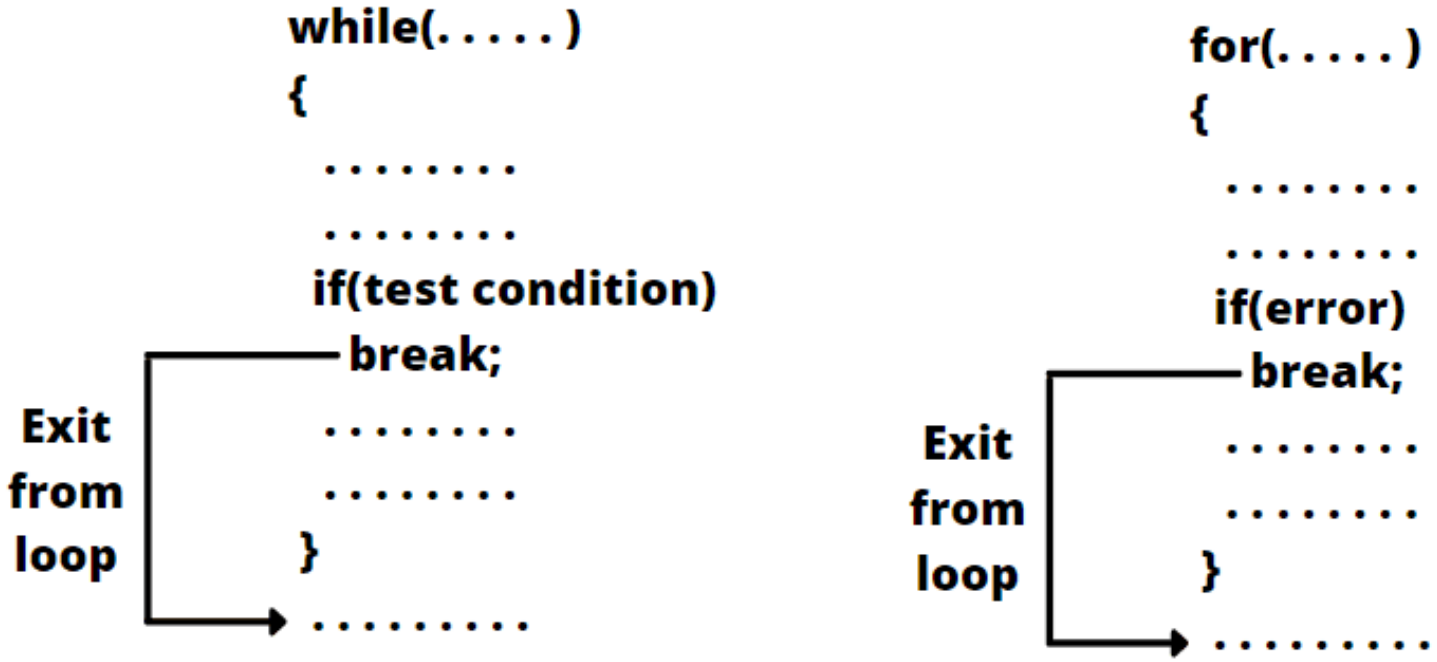


Fig: Exiting a loop with break statement

Break Statement

- _breakSample

```
public class Main {
    public static void main(String[] args) {
        for(int i = 0; i < 10; ++i) {
            if(i > 4)
                break; System.out.println(
            "i: " + i);
        }
    }
}
```

Break Statement

- _nestedLoopBreak

```
public class Main {
    public static void main(String[] args) {
        for(int i = 0; i < 2; ++i) {
            System.out.println("i: " + i);
            for(int j = 0; j < 5; ++j) {
                if (j > 2)
                    break;
                System.out.println("\tj: " + j);
            }
            System.out.println("}");
        }
    }
}
```

Continue Statement

- **Continue statement** in Java is another similar statement, like break statement that is used inside a loop to repeat the next iteration of the loop. In other words, **the continue statement stops the current iteration of loop and begins a new iteration of loop.**
- When the continue statement encounters, subsequent statements in the loop skips (i.e. not executed) at a specified condition, and the control of execution continues with the next repetition of the loop.
- Continue statement in Java **do not break out of the loop entirely**. It just jumps back to the beginning of the loop by skipping the rest of code in the loop body for the next iteration.

Continue Statement

Control goes back to the next repetition of loop

```
while(test condition )
{
  ........
  if( ......)
    continue;
  ........
}
```

(a)

```
do
{
  ........
  if( ......)
    continue;
  ........
} while(test condition);
```

(b)

Control goes back to the next repetition of loop

```
for(initialization; test condition; increment )
{
  ........
  if(..........)
    continue;
  ........
}
```

(c)

Fig: Skipping and continuing in loops

Continue Statement

- _continueSample

```
public class Main {
    public static void main(String[] args) {
        int i = 0;
        for(i = 0; i < 10; ++i) {
            if(i > 4)
                continue;
            System.out.println("\tfor i: " + i);
        }

        System.out.println("END i: " + i);
    }
}
```

Quiz

- 1. If the number of repetitions is clearly defined, what loop statement would be most readable because it expresses the initialization, condition, and increment expression on a single line?
  A. while loop
  B. for loop
  C. do-while loop
  D. if loop

Quiz

- 2. What is the best loop type if the loop body must be executed at least once, even if the condition is not met?
  A. while loop
  B. for loop
  C. do-while loop
  D. enhanced for loop

Quiz

- 3. What are the most common reasons for an infinite loop in a while loop?
  A. When the condition is initially false.
  B. When the value of a variable affecting the condition doesn't change within the loop.
  C. When a break statement is used.
  D. When a variable is declared outside the loop.

Quiz

- 4. If the outer for loop iterates 5 times and the inner for loop executes 4 times for each iteration of the outer loop, how many times will the inner loop's code be executed?
  A. 9
  B. 20
  C. 54
  D. 45

Quiz

- 5. What happens when a `continue` statement is encountered within a loop?
  A. It completely terminates the loop.
  B. It skips the remaining code in the current iteration and moves on to the next iteration.
  C. It returns to the beginning of the loop and resumes execution.
  D. It halts program execution.

03

Practice

Practice

- 请输出如下三角形
  - 要利用循环解决问题
  - (挑战)空心棱形

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Q & A

THANK YOU